Tiny College Database Project

Ashley Bui

CSCI 411 Database Theory and Design

Professor Jayantha Herath

St Cloud State University

September 2026

ER Diagram

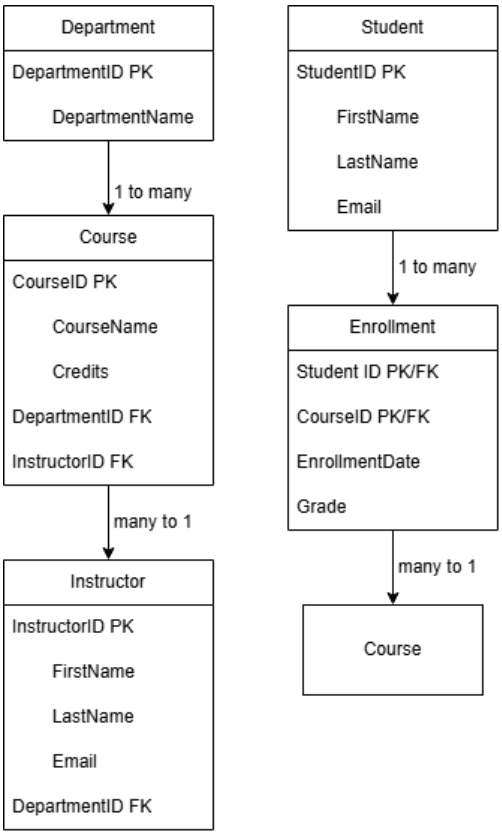


Figure 1. Tiny College ER Diagram

Normalization

1. 1NF:

Primary Keys

Table	Primary Key
Student	StudentID
Department	DepartmentID
Instructor	InstructorID
Course	CourseID
Enrollment	StudentID + CourseID

All tables contain atomic values, and there are no repeating groups. Each table has a primary key that uniquely identifies each record.

Example: Enrollment

StudentID	CourseID	EnrollmentDate	Grade
S001	C101	2026-09-01	A
S001	C102	2026-09-01	B+
S002	C101	2026-09-02	A-

Each column contains a single value, and each enrollment is uniquely identified by the combination of StudentID and CourseID.

The Tiny College database is in 1NF because all attributes contain atomic values, there are no repeating groups, and each table has a primary key.

2. 2NF:

The database is already in 1NF. To determine whether it satisfies 2NF, partial dependencies must be checked.

Example: Enrollment

The Enrollment table has a composite primary key:

(StudentID, CourseID)

Its functional dependencies are:

(StudentID, CourseID) → EnrollmentDate

(StudentID, CourseID) → Grade

Both `EnrollmentDate` and `Grade` depend on the entire composite key.

There are no partial dependencies like:

StudentID → Grade

CourseID → Grade

The other tables use single-column primary keys, so they cannot contain partial dependencies.

The Tiny College database is in 2NF because it is in 1NF and all non-key attributes depend on the entire primary key. There are no partial dependencies.

3. 3NF:

The database is already in 2NF. To satisfy 3NF, transitive dependencies must be removed.

Example: Transitive Dependency

Suppose the Course table originally stored both DepartmentID and DepartmentName.

CourseID	**CourseName**	**DepartmentID**	**DepartmentName**
C101	Database	D01	Computer Science
C102	Programming	D01	Computer Science

The dependencies would be:

CourseID → DepartmentID

DepartmentID → DepartmentName

Therefore:

CourseID → DepartmentID → DepartmentName

This is a transitive dependency.

To eliminate it, department information is stored in its own table:

Course

CourseID	**CourseName**	**Credits**	**DepartmentID**
C101	Database	3	D01
C102	Programming	3	D01

Department

DepartmentID	**DepartmentName**
D01	Computer Science

Now:

DepartmentID → DepartmentName

and the department name is stored only in the Department table.

Functional Dependencies:

Student:

StudentID → FirstName, LastName, Email

Department:

DepartmentID → DepartmentName

Instructor:

InstructorID → FirstName, LastName, Email

Course:

CourseID → CourseName, Credits, DepartmentID, InstructorID

Enrollment:

(StudentID, CourseID) → EnrollmentDate, Grade

The Tiny College database is in 3NF because it is in 2NF and contains no transitive dependencies. Each non-key attribute depends only on the primary key of its table.

4. Normalized Schema

Table	Attributes
Student	StudentID (PK), FirstName, LastName, Email
Department	DepartmentID (PK), DepartmentName
Instructor	InstructorID (PK), FirstName, LastName, Email
Course	CourseID (PK), CourseName, Credits, DepartmentID (FK), InstructorID (FK)
Enrollment	StudentID (PK, FK), CourseID (PK, FK), EnrollmentDate, Grade

This is the final normalized schema used for the SQL implementation. Primary keys uniquely identify records, while foreign keys establish relationships between the tables.

SQL Implementation

DDL – Table Creation

The database was implemented in PostgreSQL using SQL Data Definition Language (DDL). Five tables were created: Department, Instructor, Student, Course, and Enrollment. Primary keys were used to uniquely identify records, and foreign keys were used to maintain relationships between related tables. The Enrollment table uses a composite primary key consisting of StudentID and CourseID.

DML – Data Insertion

Sample data was inserted into all five tables using SQL Data Manipulation Language (DML). The database contains 2 departments, 3 instructors, 5 students, 5 courses, and 8 enrollment records. This sample data was used to test the database relationships and queries.

Sample Queries

Four sample queries were created to demonstrate that the database can retrieve information from multiple related tables.

1. Student Enrollments

Displays each student's enrolled courses, enrollment date, and grade by joining the Student, Enrollment, and Course tables.

2. Course Offerings

Displays each course, its credit value, and the department offering the course by joining the Course and Department tables.

3. Instructor Assignments

Displays the instructors and the courses they teach by joining the Instructor and Course tables.

4. Department Details

Displays each department and the courses offered by that department using a left join between the Department and Course tables.

Query Results

The following screenshots show the results of the four sample queries executed in PostgreSQL. The results demonstrate that the database tables, relationships, and queries are functioning correctly.

```
tiny_college=# SELECT
    s.StudentID,
    s.FirstName || ' ' || s.LastName AS StudentName,
    c.CourseID,
    c.CourseName,
    e.EnrollmentDate,
    e.Grade
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c ON e.CourseID = c.CourseID
ORDER BY s.StudentID, c.CourseID;
 studentid |  studentname  | courseid |        coursename         | enrollmentdate | grade
-----------+---------------+----------+---------------------------+----------------+-------
 S001      | Ashley Bui    | C101     | Database Systems          | 2026-09-01     | A
 S001      | Ashley Bui    | C102     | Programming Fundamentals  | 2026-09-01     | B+
 S002      | Daniel Nguyen | C101     | Database Systems          | 2026-09-02     | A-
 S002      | Daniel Nguyen | C103     | Data Structures           | 2026-09-02     | B
 S003      | Emily Garcia  | C102     | Programming Fundamentals  | 2026-09-03     | A
 S003      | Emily Garcia  | C201     | College Algebra           | 2026-09-03     | B+
 S004      | James Wilson  | C103     | Data Structures           | 2026-09-04     | A-
 S005      | Sophia Kim    | C201     | College Algebra           | 2026-09-04     | A
(8 rows)
```

Figure 2. Student enrollment query results

```
tiny_college=# SELECT
    c.CourseID,
    c.CourseName,
    c.Credits,
    d.DepartmentName
FROM Course c
JOIN Department d ON c.DepartmentID = d.DepartmentID
ORDER BY c.CourseID;
 courseid |        coursename         | credits |  departmentname
----------+---------------------------+---------+------------------
 C101     | Database Systems          |       3 | Computer Science
 C102     | Programming Fundamentals  |       3 | Computer Science
 C103     | Data Structures           |       3 | Computer Science
 C201     | College Algebra           |       3 | Mathematics
 C202     | Statistics                |       3 | Mathematics
(5 rows)
```

Figure 3. Course offerings query results

```
tiny_college=# SELECT
    i.InstructorID,
    i.FirstName || ' ' || i.LastName AS InstructorName,
    c.CourseID,
    c.CourseName
FROM Instructor i
JOIN Course c ON i.InstructorID = c.InstructorID
ORDER BY i.InstructorID, c.CourseID;
 instructorid | instructorname | courseid |        coursename
--------------+----------------+----------+--------------------------
 I001         | Sarah Johnson  | C101     | Database Systems
 I001         | Sarah Johnson  | C103     | Data Structures
 I002         | Michael Lee    | C102     | Programming Fundamentals
 I003         | Jennifer Smith | C201     | College Algebra
 I003         | Jennifer Smith | C202     | Statistics
(5 rows)
```

Figure 4. Instructor assignment query results

```
tiny_college=# SELECT
    d.DepartmentID,
    d.DepartmentName,
    c.CourseID,
    c.CourseName
FROM Department d
LEFT JOIN Course c ON d.DepartmentID = c.DepartmentID
ORDER BY d.DepartmentID, c.CourseID;
 departmentid |  departmentname  | courseid |        coursename
--------------+------------------+----------+--------------------------
 D01          | Computer Science | C101     | Database Systems
 D01          | Computer Science | C102     | Programming Fundamentals
 D01          | Computer Science | C103     | Data Structures
 D02          | Mathematics      | C201     | College Algebra
 D02          | Mathematics      | C202     | Statistics
(5 rows)
```

Figure 5. Department details query results

Conclusion

The Tiny College database was designed and implemented using a normalized relational database structure. The database was normalized through 3NF to reduce redundancy and maintain data integrity. PostgreSQL was used to create the tables, establish primary and foreign key relationships, insert sample data, and execute queries. The four sample queries successfully retrieved information about student enrollments, course offerings, instructor assignments, and

department details, demonstrating that the database relationships and implementation work as intended.